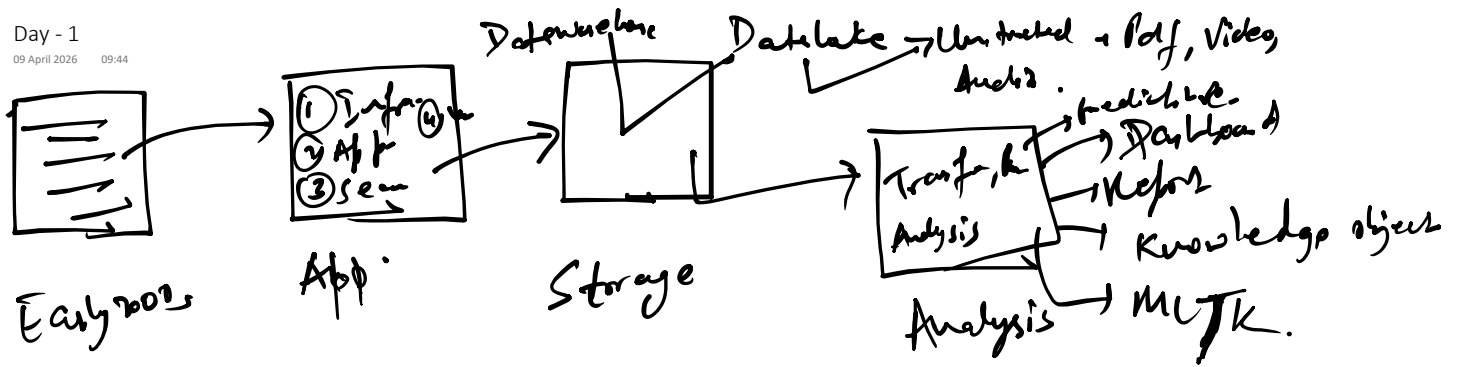




Data warehouse → Structured - CSV
Semi structured - XML, json

Monitoring → Reactive Approach.

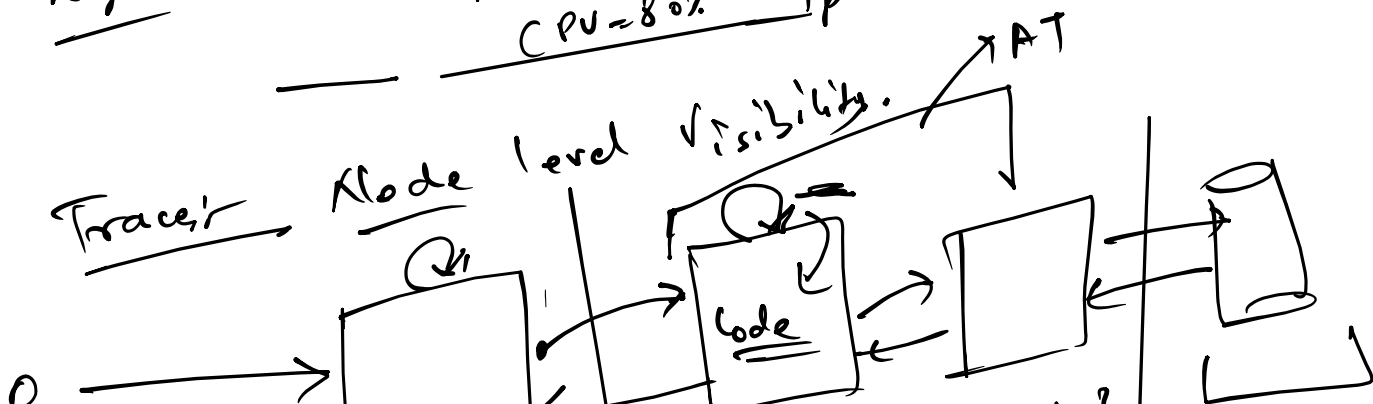
Observability → Proactive Approach.

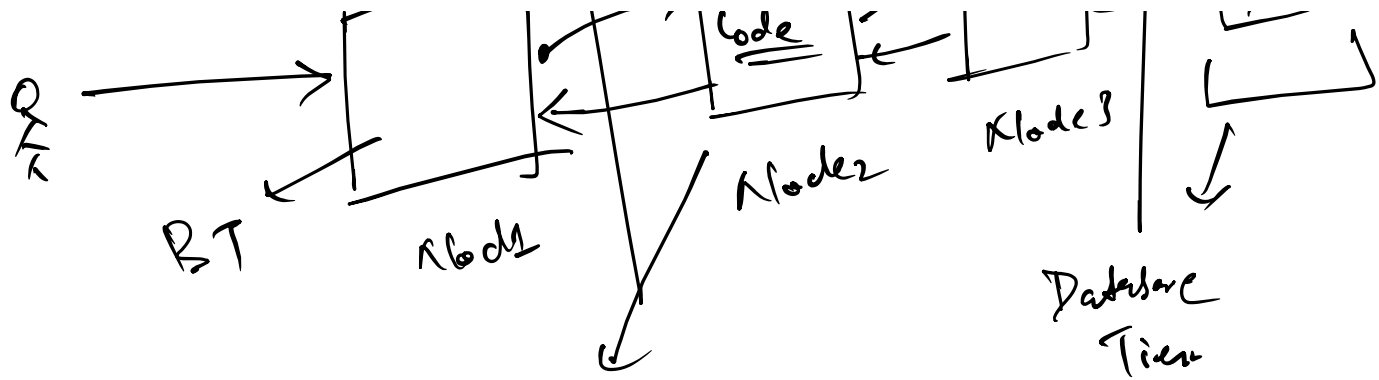
3 Pillars of observability

- ① Log
- ② Metric
- ③ Traces.

Metric's Aggregate Value.
CPU = 80% → CPU = critical

Log's Time stamp details of that Activity.
CPU = 80% IP = —



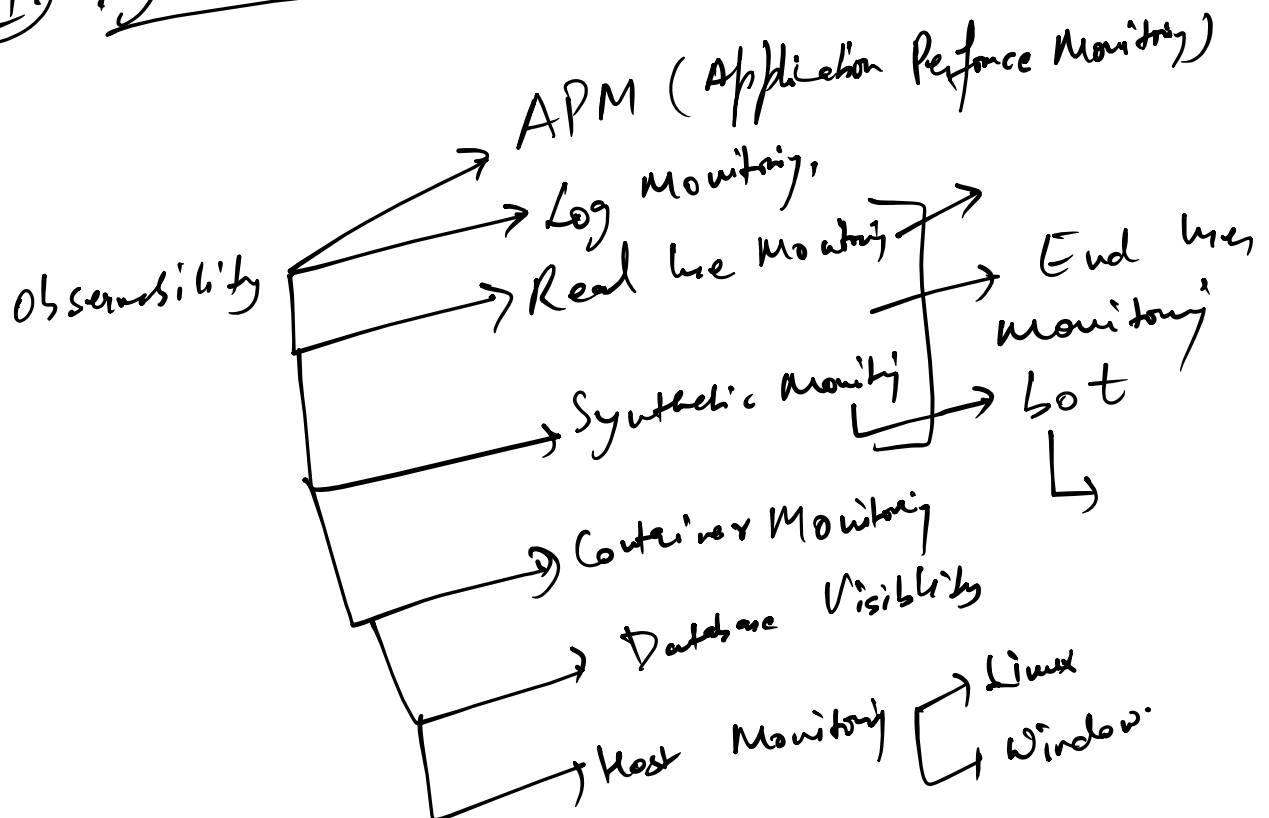


Metrix + Logs + Trace = Observability
 Trend

Observability:-

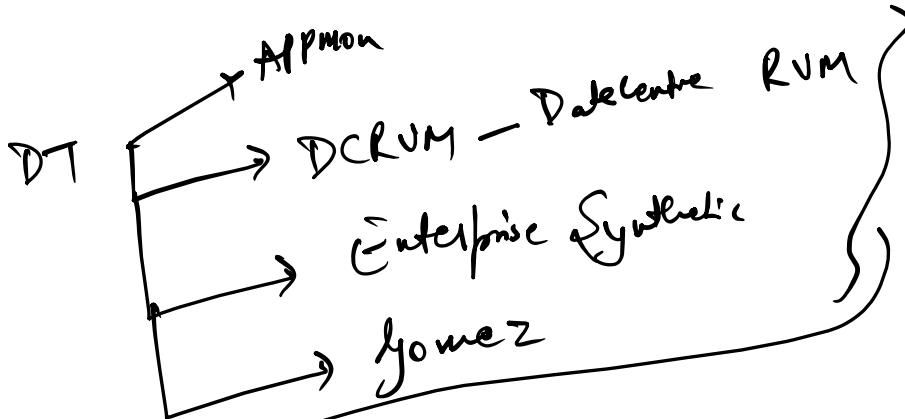
- ① Dynatrace.
- ② New Relic
- ③ AppDynamics
- ④ Splunk
- ⑤ ELK
- ⑥ Data dog. →

① Dynatrace:-

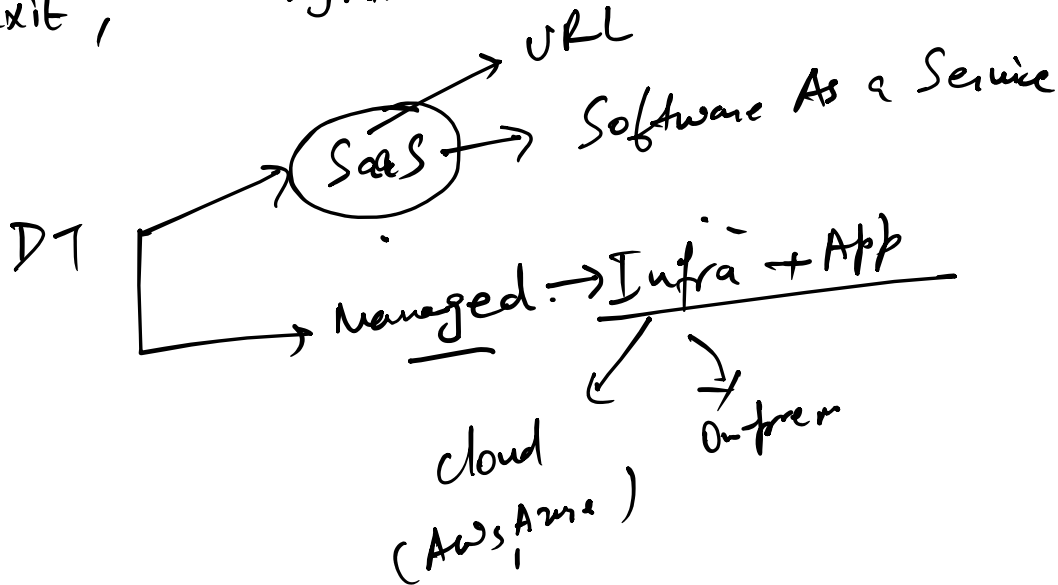


Dynatrace - Observability Tool

Around 2014

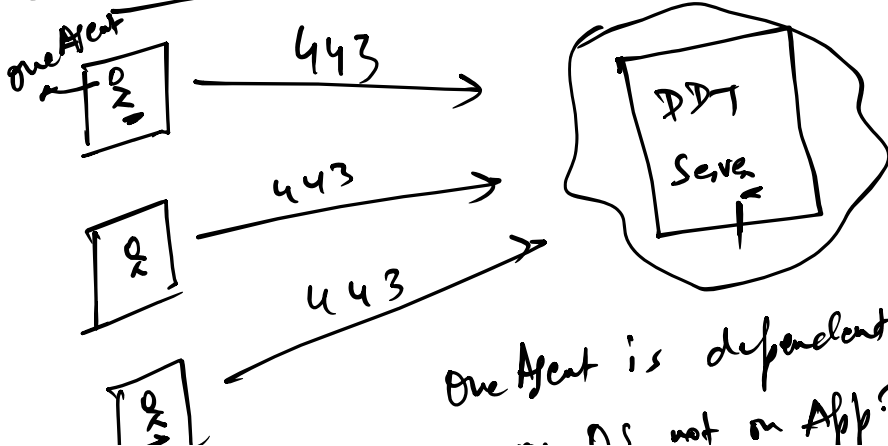


Ruxit, → Dynatrace



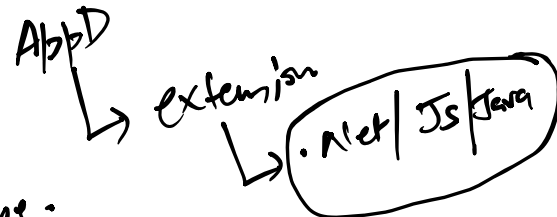
Hybrid

① Dynatrace SaaS Architecture:-



one Agent is dependent
... not on App Type.

- ① Application - Java / .Net.
- ② Infra. - Linux / Windows





Source

One Agent is deployed on OS not on App Type.

② Mission Control:-



Port 443

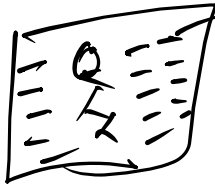


DT Server

443

Mission Control

It will always run on DT whether on-premise or



- ① License Validation
- ② Upgrade Availability
- ③ Health check of the Server

DT Server Components:-

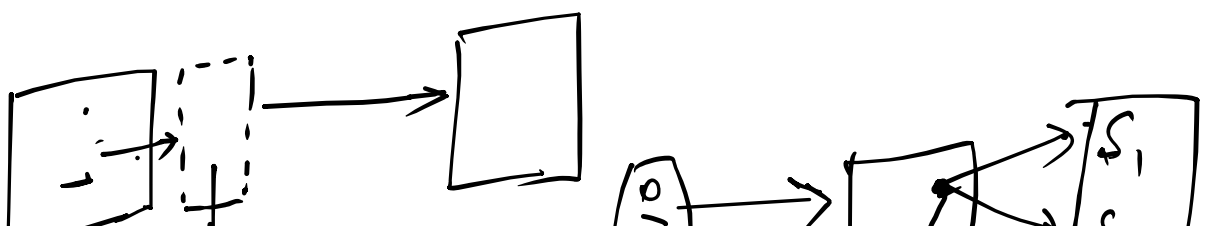
- ① Nginx - Reverse Proxy Server
- ② Elastic Search - Analyse the data.
- ③ Cassandra HyperCube - Distributed DT
- ④ DT Servers -

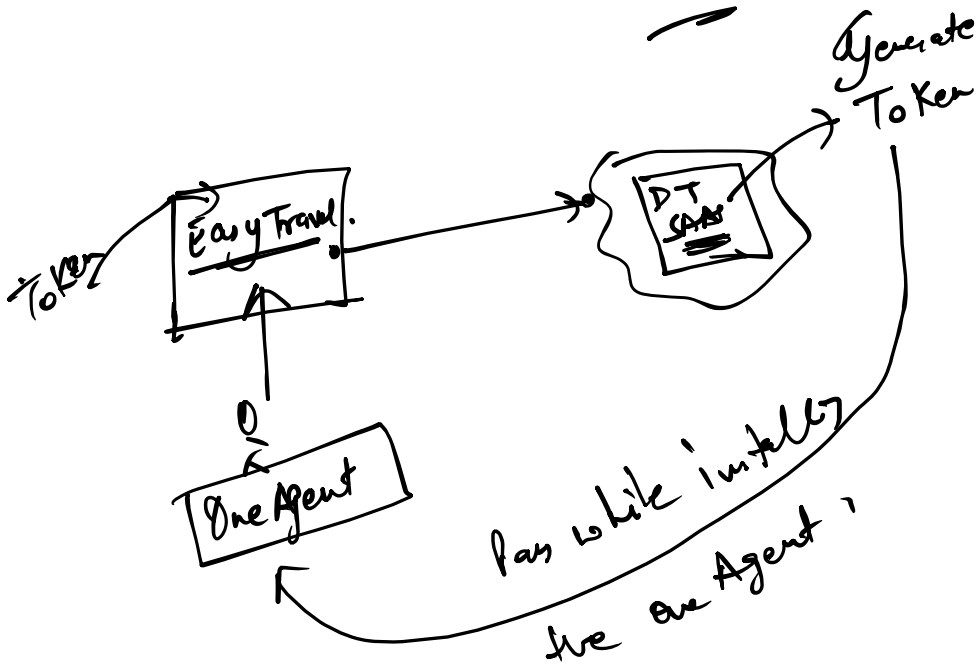
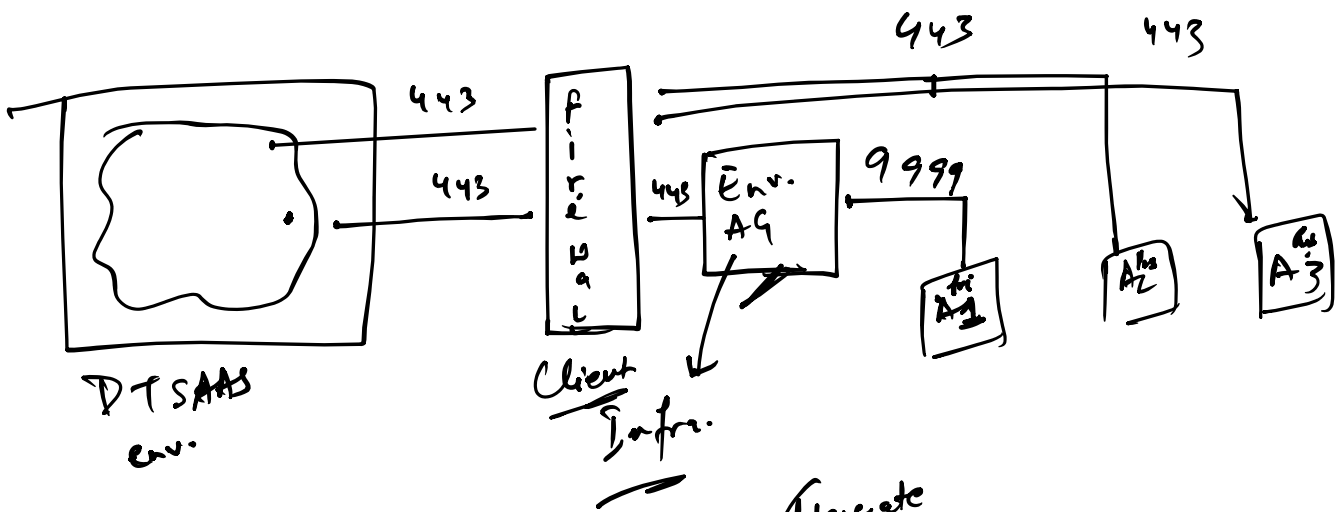
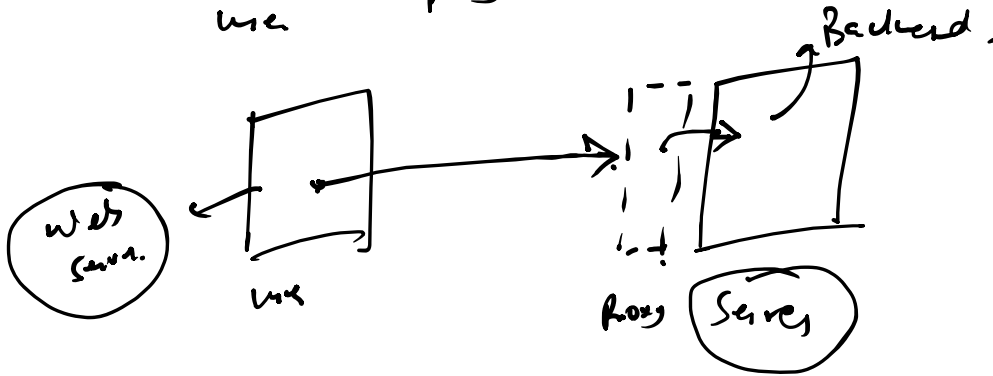
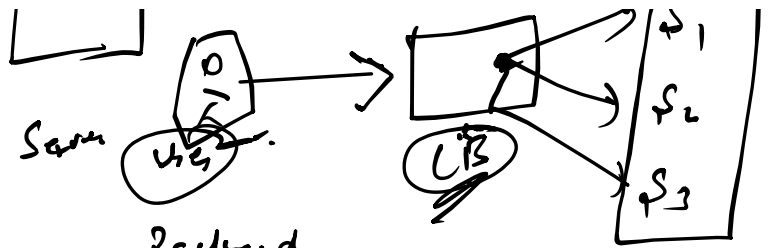
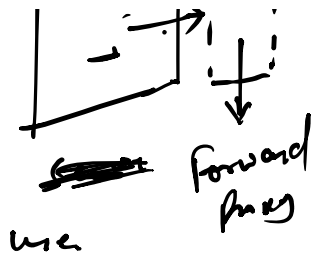
- Embedded AG
↓
SaaS env.
- ① Private App
 - ② Cloud
 - ③ Private system.

Proxy Server

→ Forward Proxy Server

→ Reverse Proxy Server



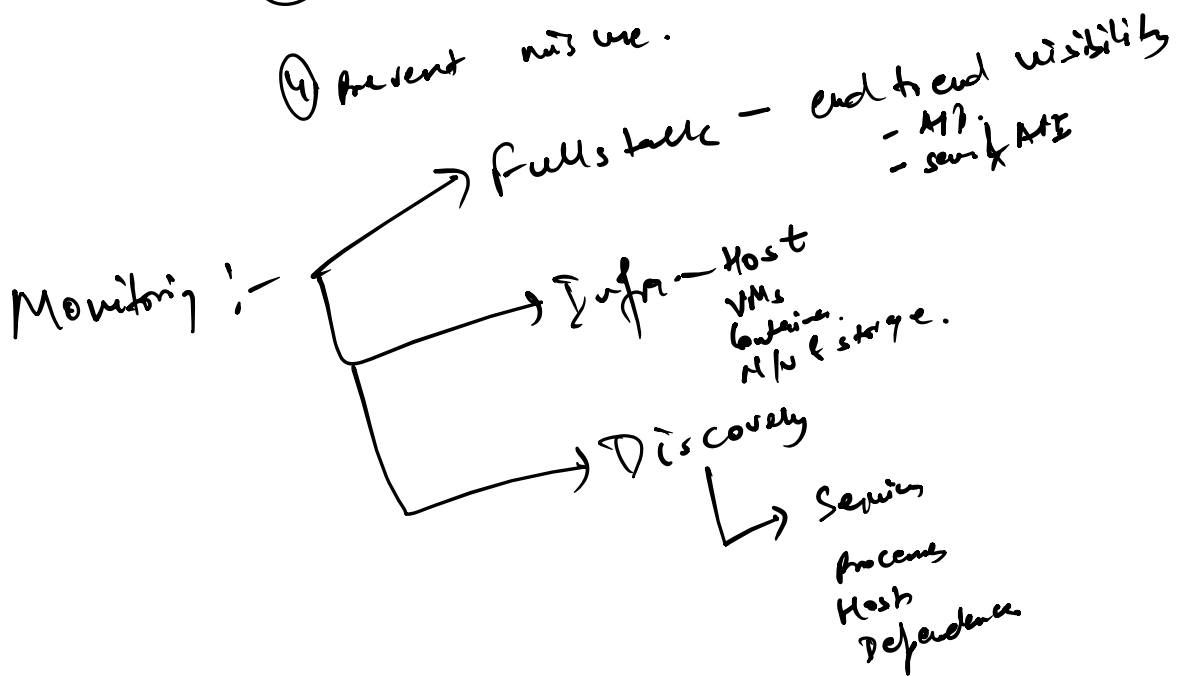


- ① API Token. (let API Accen)
- ② Pass Token Core Agent / Active Gate

② Pass Token Core Mv

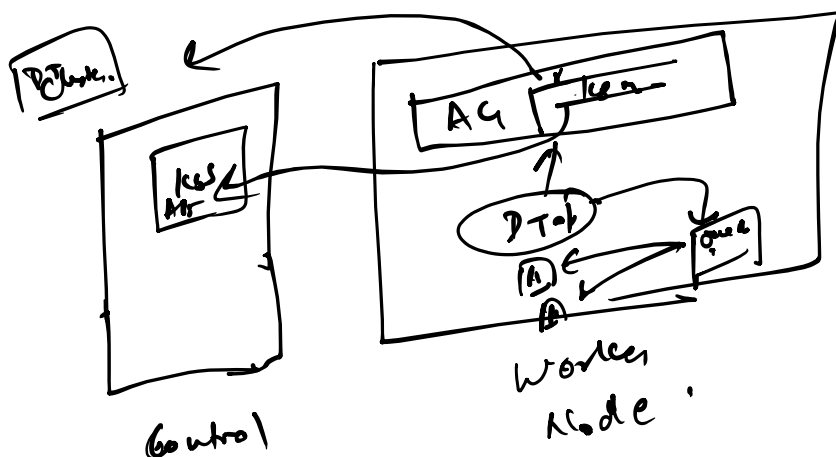
Scope = Permission Assigned to a token.
Why!

- ① Security
- ② Controlled Automation
- ③ Reduce blast radius.
 - ↳ extent of damage
- ④ prevent mis use.



① Dynatrace operation.

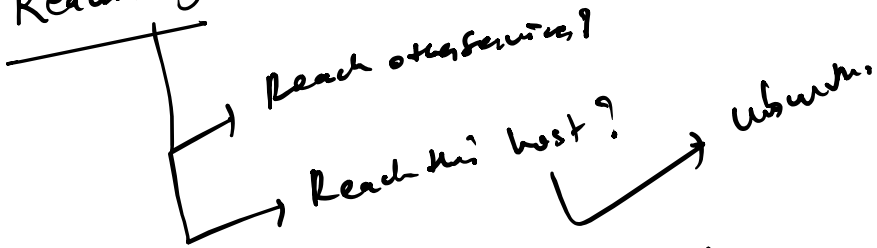
② Data ingestion - logs / Metrics / Event / Infra.



Control
Plane

Work
Node

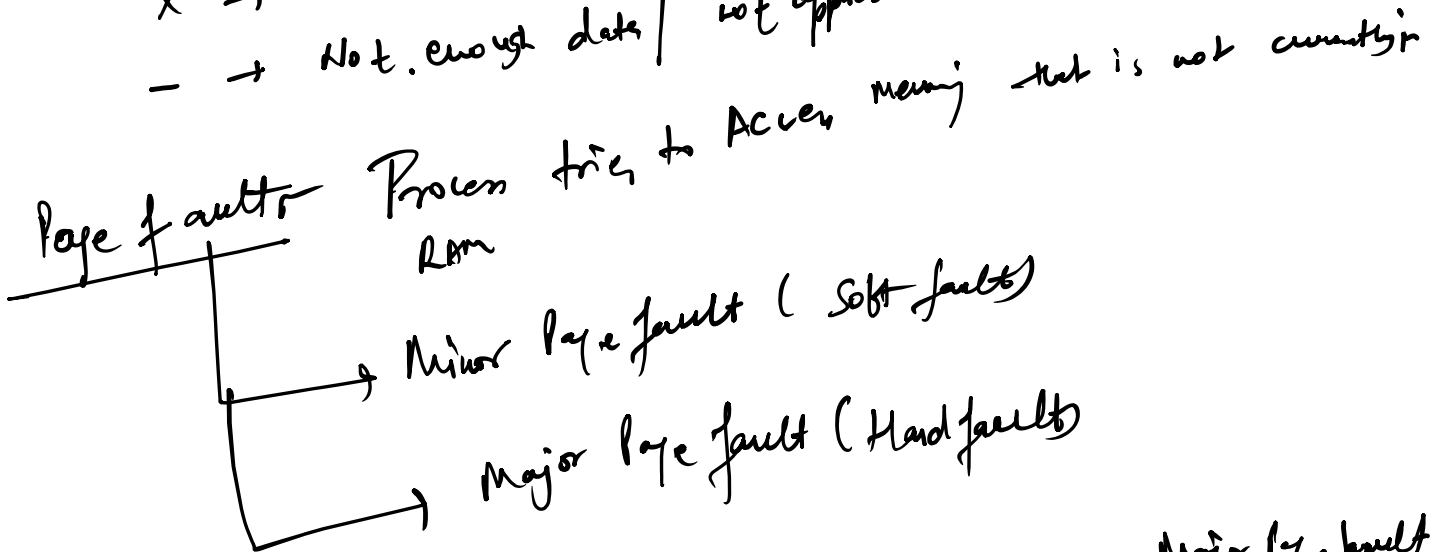
Reachability - Network connectivity status.



✓ → R/W Conn. is working/fine

X → Not reachable → Firewall, port

- → Not enough data / not applicable



App (data) → not in RAM → OS load from disk → Major page fault

App (data) → Already in RAM → just mapped → minor page fault.

Higher page fault ↑ Performance ↓

Swap space → Disk space used as extra. RAM

RAM full $\rightarrow$ move old data $\rightarrow$ swap -

need data again $\rightarrow$ bring back $\rightarrow$ page fault occur

Disk is much slower than RAM.

Heavy swap $\rightarrow$ system slow $\downarrow$

Low Ram $\propto$ OS use swap.

RAM full $\rightarrow$ Data moved to swap $\rightarrow$ App need it $\rightarrow$ Disk read $\rightarrow$ page fault $\rightarrow$ slow